

Experimental Methods For Evaluating AI Agents

Vijaya Nadendla ^{1,†} ¹ Affiliation; Independent Researcher[†] Current address: 51 Linwood Dr Monroe Township NJ 08831.

Abstract

Large language models (LLMs) are increasingly deployed as components of *AI agents* that perform multi-step analytical tasks. These agents often combine prompts, reasoning procedures, external tools, and workflow logic to produce structured analyses or decisions. Despite rapid development of such systems, experimental evaluation methods remain limited. Many studies rely on benchmarks designed for question answering or code generation, while complex analytical tasks are typically evaluated through anecdotal demonstrations or uncontrolled case studies. This paper proposes a reproducible experimental framework for evaluating AI agents performing complex analytical tasks. The framework isolates agent interventions under controlled conditions using frozen task inputs, structured generation pipelines, independent reference materials, and blinded human evaluation. Analytical outputs produced by different agents are normalized into a common structure and compared using pairwise preference judgments across multiple evaluation dimensions. We instantiate the framework using a dataset of twenty-five historical socio-technical failures drawn from domains including aerospace, finance, energy, governance, and technology. Three agent configurations are evaluated: a baseline LLM agent, a protocol-guided reasoning agent, and an evidence-augmented agent operating with independent investigation materials. The experimental design emphasizes reproducibility through frozen artifacts, hash manifests, and controlled generation procedures. The proposed framework provides a general methodology for experimentally evaluating AI agents that perform analytical reasoning tasks. Although demonstrated on structured reasoning workflows, the framework is applicable to a wide range of agent architectures, including prompt scaffolds, tool-augmented systems, and orchestration policies.

Keywords: Human–AI collaboration; epistemic governance; tacit knowledge; explicit–tacit separation; AI safety; long-horizon reasoning; category theory; sociotechnical systems

Contents

1. Introduction	3
2. AI Agents and Interventions	4
2.1. AI Agents for Analytical Tasks	4
2.2. Interventions in Agent Workflows	4
2.3. Experimental Isolation of Interventions	4
2.4. Agent Configurations Evaluated in This Study	5
3. Experimental Framework	5
3.1. Frozen Task Dataset	5
3.2. Neutral Task Inputs	5
3.3. Independent Reference Materials	6
3.4. Agent Configurations	6
3.5. Reproducible Generation Pipeline	6
3.6. Output Normalization and Blinding	7
3.7. Blinded Human Evaluation	7
3.8. Statistical Analysis	7
4. Dataset Construction	7
4.1. Case Discovery	7
4.2. Shortlisting Criteria	8
4.3. Coverage of the Solution Space	8
4.4. Exclusion Criteria	8
4.5. Case Identification and Freezing	8
4.6. Neutral Case Descriptions	8
4.7. Dataset Freezing and Reproducibility	9

1. Introduction

Large language models are increasingly deployed not as isolated text generators but as components of larger *AI agent* systems that perform structured analytical tasks. In such systems, a language model operates within a broader workflow that may include prompts, reasoning scaffolds, external tools, verification procedures, and decision logic. These agent architectures are being used for tasks such as policy analysis, incident investigation, software diagnostics, and financial risk assessment.

Despite the rapid proliferation of AI agent architectures, methods for evaluating them remain underdeveloped. Existing evaluation practices typically fall into three categories. First, many studies rely on benchmark datasets designed for question answering, reasoning puzzles, or code generation. Although useful for measuring certain capabilities, such benchmarks do not capture the open-ended analytical tasks that many AI agents are intended to perform. Second, some work evaluates agent systems through qualitative case studies or demonstrations, which can provide insight but lack experimental control and reproducibility. Third, evaluations often compare outputs produced under different prompts or agent designs without controlling for information leakage, shared inputs, or evaluator bias.

These limitations make it difficult to determine whether a proposed agent architecture genuinely improves analytical reasoning or whether observed differences arise from uncontrolled variation in prompts, inputs, or evaluation procedures.

This paper proposes a controlled experimental framework for evaluating AI agents performing complex analytical tasks. The central idea is to isolate agent interventions while holding all other aspects of the task environment constant. The framework consists of several components:

- A frozen dataset of analytical tasks constructed from historical socio-technical incidents across multiple domains.
- Neutral task descriptions that describe system structure and task intent without revealing investigative conclusions.
- Controlled agent conditions in which different agent architectures are applied to identical task inputs.
- Independent reference materials derived from authoritative investigation reports that serve as external analytical anchors.
- A reproducible generation pipeline that produces structured analytical artifacts under fixed model configurations.
- Normalization and anonymization of outputs to remove stylistic cues and enable blinded evaluation.
- Pairwise human comparison of analytical outputs across multiple evaluation dimensions.

The framework is instantiated using a dataset of twenty-five historical cases including financial crises, aerospace accidents, technology failures, and institutional breakdowns. These cases were selected because they possess rich public documentation and independent investigation reports, enabling controlled construction of analytical tasks.

Within this framework we compare three AI agent configurations. The first is a baseline LLM agent that produces analysis directly from the task description. The second is a protocol-guided agent that follows a structured reasoning workflow. The third is an evidence-augmented agent that operates with excerpts from independent investigation reports. Although the structured reasoning protocol used in this study is one particular intervention, the experimental framework is designed to evaluate a wide range of AI agent architectures.

The primary contribution of this work is methodological rather than algorithmic. We present a reproducible experimental design for evaluating AI agents on complex analytical tasks under controlled and blinded conditions. This framework can be reused to evaluate prompt strategies, reasoning protocols, tool-augmented workflows, and other forms of AI agent architecture.

2. AI Agents and Interventions

2.1. AI Agents for Analytical Tasks

In this work we use the term *AI agent* to describe a structured computational procedure that uses a language model as a reasoning component within a broader analytical workflow. Rather than producing a single response from a prompt, an AI agent may incorporate additional structure such as prompts, reasoning procedures, tool interactions, or verification steps.

Formally, an AI agent can be viewed as a mapping

$$A : X \rightarrow Y$$

where X represents a task input and Y represents a structured analytical output. The mapping is implemented through a workflow that may include intermediate artifacts, reasoning steps, or external information sources.

Examples of AI agent structures include:

- Direct LLM analysis in which a language model generates an analytical response from a task description.
- Prompt scaffolds that guide the model through structured reasoning steps.
- Protocol-guided workflows that enforce ordering constraints on the analysis process.
- Tool-augmented agents that retrieve information or execute external procedures during analysis.
- Evidence-augmented agents that incorporate excerpts from external documents or investigative materials.

In practice, modern AI systems frequently combine several of these elements, forming a reasoning pipeline in which the language model acts as one component of a larger analytical procedure.

2.2. Interventions in Agent Workflows

An *intervention* is defined as a modification to the analytical workflow of an AI agent that is intended to influence the quality, structure, or reliability of its outputs.

Examples of interventions include:

- introducing structured reasoning prompts or protocols;
- providing external evidence or retrieved documents;
- enforcing ordering constraints on analytical steps;
- requiring explicit articulation of assumptions or uncertainties;
- adding verification or diagnostic stages to the workflow.

In experimental evaluation, the objective is to determine whether a given intervention improves the analytical usefulness of the outputs produced by the agent.

2.3. Experimental Isolation of Interventions

A central challenge in evaluating AI agents is isolating the effect of a specific intervention. Differences in prompts, task descriptions, external information, or evaluation procedures can easily confound experimental results.

The framework proposed in this paper isolates interventions by holding the following elements constant across experimental conditions:

- the task dataset;
- the task input descriptions;
- the underlying language model and generation configuration;
- the evaluation procedure and scoring rubric.

Only the agent workflow itself is varied. This allows observed differences in analytical outputs to be attributed to the intervention under study rather than to unrelated experimental variation.

2.4. Agent Configurations Evaluated in This Study

To demonstrate the proposed experimental framework, we evaluate three agent configurations that differ in their analytical workflow:

- **Baseline agent (B)**: the language model generates analysis directly from the task description.
- **Protocol-guided agent (P)**: the language model follows a structured reasoning workflow that constrains the order and form of analytical steps.
- **Evidence-augmented agent (I)**: the language model performs analysis using excerpts derived from independent investigation reports.

These configurations are used as an instantiation of the experimental framework. The methodology itself is designed to evaluate a broad class of AI agent architectures, including prompt scaffolds, reasoning protocols, tool-augmented agents, and other structured workflows.

3. Experimental Framework

This section describes the experimental framework used to evaluate AI agent workflows under controlled conditions. The framework is designed to isolate the effect of agent interventions while ensuring reproducibility, comparability of outputs, and independence of evaluation.

The experimental design consists of the following components:

1. construction of a frozen task dataset;
2. creation of neutral task inputs that describe the system under analysis without revealing investigative conclusions;
3. definition of multiple agent configurations that differ only in their analytical workflow;
4. generation of analytical artifacts using a controlled model configuration;
5. normalization and anonymization of outputs;
6. blinded human evaluation using pairwise comparison;
7. statistical analysis of comparative preferences.

Together these elements provide a reproducible procedure for experimentally evaluating AI agents performing complex analytical tasks.

3.1. Frozen Task Dataset

The experimental task set consists of twenty-five historical cases drawn from multiple socio-technical domains, including aerospace accidents, financial failures, energy infrastructure incidents, technology failures, and institutional governance breakdowns.

Cases were selected through an iterative discovery and screening process designed to produce a diverse and information-rich evaluation set. Each candidate case was required to satisfy the following criteria:

- the event involved a major failure or systemic breakdown;
- sufficient public documentation existed to describe the system structure and operational context;
- at least one credible independent investigation or assessment report was available;
- the case was suitable for comparative analytical reconstruction.

The final set of twenty-five cases was assigned stable identifiers (C01–C25) and frozen in the file `case_list_v1.txt`. This file serves as the root index for all subsequent experimental artifacts.

3.2. Neutral Task Inputs

For each case, a neutral description of the system and task context was constructed. These descriptions focus on system architecture, operational processes, and organizational structure while intentionally excluding investigative conclusions or causal findings.

The construction procedure consisted of:

1. writing a short system description (target length ≤ 150 words);

2. removing statements that reveal the outcome of the investigation;
3. excluding conclusions drawn by investigative reports;
4. preserving only information necessary to understand the system structure and task intent.

The resulting descriptions were frozen in the artifact `neutral_case_descriptions_v1.txt`. During generation, these descriptions are converted into individual task input files (`Cxx_case_input.txt`) corresponding to each case.

3.3. Independent Reference Materials

To provide an external analytical anchor, authoritative investigation sources were identified for each case. Sources were selected according to a credibility hierarchy that prioritizes formal investigative authority:

1. court findings or judicial determinations;
2. congressional or parliamentary investigation reports;
3. regulatory authority reports;
4. independent investigation boards or commissions.

For each case, the most authoritative available source was designated as *Primary A*. When a second broad investigation source was available, an alternate source was designated as *Primary B* for sensitivity analysis. Supplemental sources were included when they provided additional technical or operational detail not covered in the primary report.

Relevant excerpts describing system architecture, operational processes, decision structures, causal mechanisms, and investigative findings were manually extracted from these sources and stored as frozen artifacts (`Cxx_refA_extract.txt`, `Cxx_refB_extract.txt`).

3.4. Agent Configurations

Three AI agent configurations are evaluated within the framework:

- **Baseline agent (B)**: the language model generates analysis directly from the neutral task description.
- **Protocol-guided agent (P)**: the model follows a structured reasoning workflow that constrains the order and form of analytical artifacts.
- **Evidence-augmented agent (I)**: the model performs analysis using excerpts from independent investigation materials.

All agent configurations operate on identical task inputs and use the same underlying model configuration. The only difference between conditions is the analytical workflow applied by the agent.

3.5. Reproducible Generation Pipeline

Analytical outputs are generated using a controlled execution pipeline implemented in a reproducible notebook environment.

For each case identifier C_{xx} , the pipeline constructs a task input file:

`Cxx_case_input.txt`

Generation is then performed for each agent configuration, producing raw analytical artifacts:

- `Cxx_B_raw.txt`
- `Cxx_P_raw.txt`
- `Cxx_I_refA_raw.txt`
- `Cxx_I_refB_raw.txt`

All generations are executed under a fixed model configuration specified in `model_config_v1.json`. Each generated output is stored together with a SHA-256 hash and a generation log entry recording the case identifier, generation condition, model configuration, and timestamp.

3.6. Output Normalization and Blinding

Raw outputs produced by different agents may vary in length, structure, and stylistic presentation. To enable fair comparison, outputs are normalized into a common analytical template and anonymized prior to evaluation.

Each normalized document is assigned a random identifier (e.g., D0001, D0002) that hides the generation condition from human evaluators. The mapping between document identifiers and generation conditions is stored in a separate key file not accessible to raters.

3.7. Blinded Human Evaluation

Human raters evaluate outputs using pairwise comparison. Rather than scoring individual documents in isolation, raters are presented with two analytical outputs describing the same case and asked to indicate which analysis is closer to a high-quality analytical reconstruction.

Each comparison is evaluated along multiple dimensions, including:

- analytical clarity;
- structural reasoning quality;
- diagnostic usefulness;
- overall analytical usefulness.

Document order is randomized to prevent systematic position bias.

3.8. Statistical Analysis

Evaluation results are analyzed using pairwise preference statistics. Directional preferences between agent conditions are aggregated across cases and raters.

The primary measure is the preference rate between conditions. Statistical significance of observed preference differences is assessed using non-parametric tests such as the binomial sign test.

This analysis allows the experimental framework to quantify whether a given agent intervention leads to systematically preferred analytical outputs.

4. Dataset Construction

The experimental framework requires a task dataset that supports controlled analytical reconstruction while remaining independent of the evaluation procedures applied by the agents. To satisfy this requirement, a dataset of twenty-five historical socio-technical cases was constructed and frozen prior to generation and evaluation.

The objective of the dataset construction process was not to create a statistically representative sample of failures, but rather to assemble an information-rich collection of cases that support structured analysis and independent verification.

4.1. Case Discovery

Candidate cases were identified through an iterative discovery process covering multiple socio-technical domains, including:

- finance and financial regulation;
- aerospace and spaceflight;
- energy infrastructure;
- transportation systems;
- technology and software systems;
- corporate governance and institutional systems.

The initial discovery stage prioritized breadth of coverage rather than immediate inclusion. Candidate events were drawn from publicly documented historical failures and controversies known to involve complex system interactions, organizational decision-making, or governance breakdowns.

4.2. Shortlisting Criteria

Candidate cases were then screened against four requirements.

1. **Major system failure or breakdown**

The event must involve substantial consequences such as financial loss, loss of life, mission failure, systemic disruption, or significant governance failure.

2. **Publicly describable system setup**

Sufficient public documentation must exist to describe the system architecture, operational processes, and organizational structure without relying on the conclusions of investigative reports.

3. **Availability of independent assessment**

The case must have at least one credible external investigation or assessment source, such as court findings, congressional investigations, regulatory reports, or independent inquiry commissions.

4. **Suitability for analytical reconstruction**

The case must support reconstruction of system behavior and decision processes in a way that allows meaningful comparison between different analytical workflows.

4.3. Coverage of the Solution Space

During shortlisting, cases were intentionally selected to span a wide solution space rather than cluster around a single failure type.

The final dataset includes cases characterized by:

- mechanistic engineering failures;
- socio-technical governance failures;
- financial and institutional breakdowns;
- regulatory and oversight failures;
- software and information-system failures.

This diversity allows the experimental framework to evaluate whether different AI agent workflows perform differently across classes of analytical problems.

4.4. Exclusion Criteria

Candidate cases were excluded if they failed to meet one or more of the following conditions:

- no credible independent investigation source could be identified;
- publicly available documentation of system structure was too limited;
- the case was redundant with another selected case;
- the case remained too unresolved or factually unstable to support a frozen analytical task.

4.5. Case Identification and Freezing

After iterative screening, twenty-five cases were selected and assigned stable identifiers:

$$C01, C02, \dots, C25$$

The finalized case list was frozen in the artifact

`case_list_v1.txt`

which serves as the root index for all experimental inputs and generated artifacts.

Each case identifier is subsequently used to generate associated task inputs, reference materials, and analytical outputs throughout the experimental pipeline.

4.6. Neutral Case Descriptions

For each case, a neutral system description was created to serve as the primary task input for agent analysis.

The construction procedure consisted of:

1. writing a concise description of the system and operational context (target length ≤ 150 words);
2. focusing on system architecture, operational processes, and task intent;
3. removing investigative conclusions and causal findings;
4. excluding language that directly reveals the outcome of the event.

These descriptions were frozen in the artifact

`neutral_case_descriptions_v1.txt`

and later converted into individual case input files during generation.

The purpose of this neutralization procedure is to ensure that all agent configurations operate from the same initial information state, preventing direct leakage of investigative conclusions into the analytical task.

4.7. Dataset Freezing and Reproducibility

To ensure reproducibility, all dataset artifacts were frozen prior to model generation and evaluation. Each artifact was recorded together with a cryptographic hash in a freeze manifest.

The frozen artifacts include:

- `case_list_v1.txt`
- `neutral_case_descriptions_v1.txt`
- `prompt_baseline_v1.txt`
- `prompt_protocol_v1.txt`
- `prompt_independent_v1.txt`
- `protocol_specification_v1.txt`
- `model_config_v1.json`
- `case_id_mapping_v1.csv`

Hashes for these artifacts are recorded in

`case_freeze_manifest_v1.txt`

which enables verification that the dataset and experimental configuration remain unchanged across experimental runs.